

MEMORIA DE PRÁCTICAS

4⁰ Vuelta Atrás

VUELTA ATRÁS (SUDOKU)

En un sudoku disponemos de un tablero de tamaño 9x9 en cuyas celdas se pueden situar valores entre 1 y 9. A su vez, el tablero queda dividido en 9 subtableros de tamaño 3x3. El valor de algunas celdas está fijado inicialmente. El juego consiste en completar los valores de las demás celdas de modo que se cumplan las siguientes reglas:

- 1) un mismo valor no puede aparecer más de una vez en la misma fila del tablero,
- 2) un mismo valor no puede aparecer más de una vez en la misma columna del tablero,
- 3) y un mismo valor no puede aparecer más de una vez en el mismo subtablero.

OBJETIVOS DE LA PRÁCTICA:

a) Implementar un algoritmo, mediante la técnica de vuelta atrás, que obtenga todas las soluciones posibles para una configuración inicial de sudoku.

b) Evaluar experimentalmente la corrección de su implementación, tal como se explica posteriormente.

Para resolver este problema, definiremos una clase `TableroSudoku`, de modo que un objeto instancia de la clase representa una instancia del problema.

Los siguientes métodos auxiliares deberán ser implementados por el alumno:

- `boolean estáEnFila(int fila, int valor)`, que devuelve `true` si el valor `valor` ya aparece en alguna posición de la fila `fila`.
- `boolean estáEnColumna(int columna, int valor)`, que devuelve `true` si el valor `valor` ya aparece en alguna posición de la columna `columna`.
- `boolean estáEnSubtablero (int fila, int columna, int valor)`, que devuelve `true` si el valor `valor` ya aparece en alguna posición dentro del subtablero al que corresponde la posición dada por fila y columna.
- `boolean sePuedePonerEn(int fila, int columna, int valor)`, que devuelve `true` si el valor `valor` puede colocarse en la celda dada por fila y columna.

<<Java Class>>	
TableroSudoku	
(default package)	
MAXVALOR: int	
FILAS: int	
COLUMNAS: int	
r: Random	
celdas: int[][]	
TableroSudoku()	
TableroSudoku(TableroSudoku)	
TableroSudoku(String)	
clone(): Object	
equals(Object): boolean	
toString(): String	
estáLibre(int,int): boolean	
numeroDeLibres(): int	
numeroDeFijos(): int	
estáEnFila(int,int): boolean	
estáEnColumna(int,int): boolean	
estáEnSubtablero(int,int,int): boolean	
sePuedePonerEn(int,int,int): boolean	
resolverTodos(List<TableroSudoku>,int,int): void	
nuevaPosicion(int,int): int[]	
resolverTodos(): List<TableroSudoku>	
main(String[]): void	

La resolución del problema, mediante la técnica de vuelta atrás, se realizará completando `void resolverTodos (List<TableroSudoku> soluciones , int fila, int columna)`, que añada a la lista `soluciones` todas las soluciones válidas para el tablero que se pueden obtener rellendo las celdas a partir de la fila y la columna inclusive.

Como el código es bastante extenso junto con todos los métodos auxiliares ya dados, pondré al final todo el código pero iré comentando primero método a método mi resolución para aclarar cosas.

En primer lugar, deberemos crear un método que devuelva si el valor dado está en la fila metida como parámetro o no, para ello recorreremos de izquierda a derecha nuestra tabla celda [][] valor que indica 0 si una posición esta vacía o el número de 1 a 9 en caso de que ya hayamos añadido algo. Si recorre la fila y no está dicho número devolverá false.

```
protected boolean estaEnFila(int fila, int valor) {  
    for (int i = 0; i < COLUMNAS; i++) {  
        if (celdas[fila][i] == valor) {  
            return true;  
        }  
    }  
    return false;  
}
```

Muy similar es el siguiente método al que nos enfrentamos, simplemente ahora deberemos ver si existe el valor en la misma columna recorriéndola de arriba abajo en nuestra tabla celda [][].

```
protected boolean estaEnColumna(int columna, int valor) {  
    for (int i = 0; i < FILAS; i++) {  
        if (celdas[i][columna] == valor) {  
            return true;  
        }  
    }  
    return false;  
}
```

En el siguiente método se nos pide ver si el valor dado pertenece al subtablero donde se encuentra la posición (fila, columna) dadas. Un sudoku se puede subdividir en 9 tableros de 3x3 en los cuáles tampoco hay números repetidos por lo que debemos localizar en cuál de los 9 tableros estamos y después ver si en dicho tablero se repite el valor que queremos añadir.

```
protected boolean estaEnSubtablero(int fila, int columna, int valor) {  
    for (int i = (3*(fila / 3)); i < 3*(1 + (fila / 3)); i++) {  
        for (int j = (3*(columna / 3)); j < 3*(1 + columna / 3); j++) {  
            if (celdas[i][j] == valor) {  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

Por último, las tres condiciones que hemos formalizado son las que debe no cumplir cada valor que queramos meter en nuestra posición (fila, columna) en el sudoku, por ello en vez de llamar a las tres, creamos un nuevo método que comprueba las tres condiciones. Si alguna de ellas se cumple, es decir, ya existe un valor en la fila, columna o subtablero, no se podrá poner dicho número.

```
protected boolean sePuedePonerEn(int fila, int columna, int valor) {

    return !estaEnColumna(columna, valor) && !estaEnFila(fila,
valor) && !estaEnSubtablero(fila, columna, valor);

}
```

Ya por último nos queda el algoritmo de vuelta atrás, para ello primero voy a comentar un método que he creado para obtener el siguiente hueco en el sudoku, devolviendo un array de tamaño 2 con la fila en la posición 0 y la columna en la 1.

Recorriendo la tabla desde la última posición donde metimos valor de izquierda a derecha y de arriba abajo, en el momento en el que encuentra una posición libre devuelve esta. En caso de que no haya más posiciones accesibles en la tabla devolveremos null. Esta función es aquella que me **genera el candidato** para el algoritmo de vuelta atrás.

```
private int[] nuevaPosicion(int fila, int columna) {

    int filas = fila;
    int columnas = columna;

    while (filas < FILAS) {

        if (estaLibre(filas, columnas)) {
            int[] aux = new int[2];
            aux[0] = filas;
            aux[1] = columnas;
            return aux;
        }

        columnas++;

        if (columnas == COLUMNAS) {
            columnas = 0;
            filas++;
        }
    }

    return null;
}
```

Por último comentemos la función de resolverTodos, donde llamaremos a nuestra función nuevaPosicion como primer paso. Si detecta que el tablero actual está completo, es decir que no quedan más huecos en la tabla, añadiremos una nueva solución a la lista soluciones de la forma requerida en el enunciado.

En cambio, si no está completa dicha tabla tomaremos los parámetros fila y columna de la siguiente posición libre y realizaremos un bucle intentando añadir un valor comprendido del 1 al 9 que cumpla las tres condiciones definidas anteriormente. Si así lo hace, cambiaremos nuestra tabla añadiendo el valor k en la posición y llamaremos de nuevo a la función con el nuevo estado de la tabla y la posición metida como la última añadida haciendo que en el caso de vuelta atrás dejemos vacía la nueva casilla para buscar nuevas soluciones.

```
protected void resolverTodos(List<TableroSudoku> soluciones, int fila, int
columna) {

    int[] next = nuevaPosicion(fila, columna);

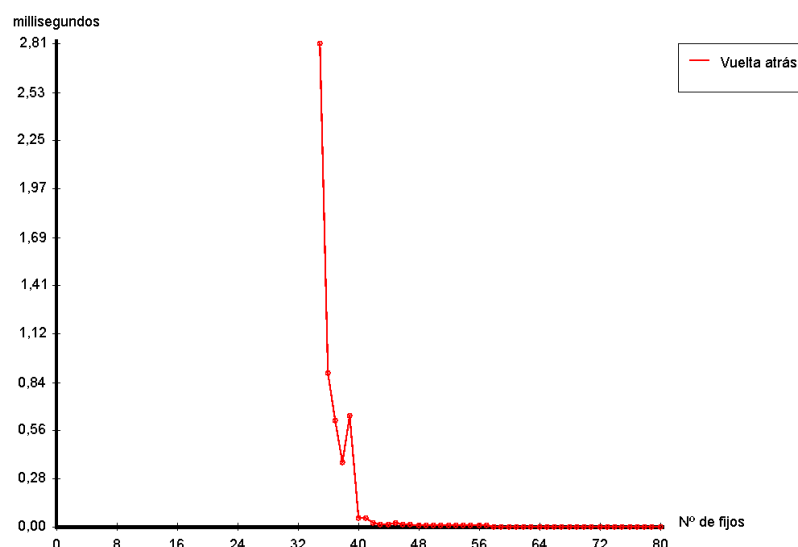
    if (next == null) {
        soluciones.add(new TableroSudoku(this));
    } else {

        int fil = next[0];
        int col = next[1];

        for (int k = 1; k <= MAXVALOR; k++) {
            if (sePuedePonerEn(fil, col, k)) {
                celdas[fil][col] = k;
                resolverTodos(soluciones, fil, col);
            }
        }

        celdas[fil][col] = 0;
    }
}
```

Gracias a las clases auxiliares dadas, hemos podido realizar la corrección de nuestro algoritmo. Además, la clase `TestsTiempos` nos muestra la evolución del tiempo requerido en función del número de valores que añadamos de partida al sudoku y me parece muy interesante su caída a partir de un número de elementos superior a la mitad de los que tenemos que añadir.



```

public class TableroSudoku implements Cloneable {

    // constantes relativas al nº de filas y columnas del tablero
    protected static final int MAXVALOR = 9;
    protected static final int FILAS = 9;
    protected static final int COLUMNAS = 9;

    protected static Random r = new Random();

    protected int celdas[][]; // una celda vale cero si est\u00E1 libre.

    public TableroSudoku() {
        celdas = new int[FILAS][COLUMNAS]; // todas a cero.
    }

    // crea una copia de su par\u00E1metro
    public TableroSudoku(TableroSudoku uno) {
        TableroSudoku otro = (TableroSudoku) uno.clone();
        this.celdas = otro.celdas;
    }

    // crear un tablero a partir de una configuraci\u00D3n inicial (las
    celdas
    // vac\u00EDas
    // se representan con el caracter ".".
    public TableroSudoku(String s) {
        this();
        if (s.length() != FILAS * COLUMNAS) {
            throw new RuntimeException("Construcci\u00D3n de sudoku no
v\u00E1lida.");
        } else {
            for (int f = 0; f < FILAS; f++)
                for (int c = 0; c < COLUMNAS; c++) {
                    Character ch = s.charAt(f * FILAS + c);
                    celdas[f][c] = (Character.isDigit(ch) ?
Integer.parseInt(ch.toString()) : 0);
                }
        }
    }

    /*
     * Realizar una copia en profundidad del objeto
     */
    @see java.lang.Object#clone()
    public Object clone() {
        TableroSudoku clon;
        try {
            clon = (TableroSudoku) super.clone();
            clon.celdas = new int[FILAS][COLUMNAS];
            for (int i = 0; i < celdas.length; i++)
                System.arraycopy(celdas[i], 0, clon.celdas[i], 0,
celdas[i].length);
        } catch (CloneNotSupportedException e) {
            clon = null;
        }
        return clon;
    }
}

```

```

/*
 * Igualdad para la clase
 *
 * @see java.lang.Object#equals()
 */
public boolean equals(Object obj) {
    if (obj instanceof TableroSudoku) {
        TableroSudoku otro = (TableroSudoku) obj;
        for (int f = 0; f < FILAS; f++)
            if (!Arrays.equals(this.celdas[f], otro.celdas[f]))
                return false;
        return true;
    } else
        return false;
}

public String toString() {
    String s = "";

    for (int f = 0; f < FILAS; f++) {
        for (int c = 0; c < COLUMNAS; c++)
            s += (celdas[f][c] == 0 ? "." : String.format("%d",
celdas[f][c]));
        }
        return s;
    }

    // devuelva true si la celda del tablero dada por fila y columna
    // est
    // vac
    protected boolean estaLibre(int fila, int columna) {
        return celdas[fila][columna] == 0;
    }

    // devuelve el número de casillas libres en un sudoku.
    protected int numeroDeLibres() {
        int n = 0;
        for (int f = 0; f < FILAS; f++)
            for (int c = 0; c < COLUMNAS; c++)
                if (estaLibre(f, c))
                    n++;
        return n;
    }

    protected int numeroDeFijos() {
        return FILAS * COLUMNAS - numeroDeLibres();
    }

    // Devuelve true si @valor ya esta en la fila @fila.
    protected boolean estaEnFila(int fila, int valor) {
        for (int i = 0; i < COLUMNAS; i++) {
            if (celdas[fila][i] == valor) {
                return true;
            }
        }
        return false;
    }
}

```

```

// Devuelve true si @valor ya esta en la columna @columna.
protected boolean estaEnColumna(int columna, int valor) {

    for (int i = 0; i < FILAS; i++) {
        if (celdas[i][columna] == valor) {
            return true;
        }
    }

    return false;
}

// Devuelve true si @valor ya esta en subtablero al que pertenece @fila
y
// @columna.
protected boolean estaEnSubtablero(int fila, int columna, int valor) {

    for (int i = (3 * (fila / 3)); i < 3 * (1 + (fila / 3)); i++) {
        for (int j = (3 * (columna / 3)); j < 3 * (1 + columna /
3); j++) {
            if (celdas[i][j] == valor) {
                return true;
            }
        }
    }

    return false;
}

// Devuelve true si se puede colocar el @valor en la @fila y @columna
dadas.
protected boolean sePuedePonerEn(int fila, int columna, int valor) {
    return !estaEnColumna(columna, valor) && !estaEnFila(fila,
valor) && !estaEnSubtablero(fila, columna, valor);
}

protected void resolverTodos(List<TableroSudoku> soluciones, int fila,
int columna) {

    int[] next = nuevaPosicion(fila, columna);

    if (next == null) {
        soluciones.add(new TableroSudoku(this));
    } else {

        int fil = next[0];
        int col = next[1];

        for (int k = 1; k <= MAXVALOR; k++) {
            if (sePuedePonerEn(fil, col, k)) {
                celdas[fil][col] = k;
                resolverTodos(soluciones, fil, col);
            }
        }

        celdas[fil][col] = 0;
    }
}

```



```

private int[] nuevaPosicion(int fila, int columna) {

    int filas = fila;
    int columnas = columna;

    while (filas < FILAS) {

        if (estalibre(filas, columnas)) {
            int[] aux = new int[2];
            aux[0] = filas;
            aux[1] = columnas;
            return aux;
        }

        columnas++;

        if (columnas == COLUMNAS) {
            columnas = 0;
            filas++;
        }
    }

    return null;
}

public List<TableroSudoku> resolverTodos() {
    List<TableroSudoku> sols = new LinkedList<TableroSudoku>();
    resolverTodos(sols, 0, 0);
    return sols;
}

public static void main(String arg[]) {
    TableroSudoku t = new TableroSudoku(

        ".4....36263.941...5.7.3.....9.3751..3.48.....17..62...716.9..2...96..
        .....312..9.");
    List<TableroSudoku> lt = t.resolverTodos();
    System.out.println(t);
    System.out.println(lt.size());
    for (Iterator<TableroSudoku> i = lt.iterator(); i.hasNext();) {
        TableroSudoku ts = i.next();
        System.out.println(ts);
    }
}
}

```